Fine-tune LLM for Mechanised Proof Automation

1. Background and Motivation

Mechanised theorem provers are a well-established and indispensable tool for reproducibly verifying the correctness of formal specifications. Isabelle and Rocq have become de-facto requirements for conference submissions in formal methods, while Lean has gained significant traction in mathematics, physics, and theoretical computer science following Terence Tao's influential work on formalised proofs. As specifications grow increasingly complex, the effort required to manually write out proofs has become a bottleneck in scientific discovery. This is where artificial intelligence can help. By combining the ability of large language models (LLMs) to predict proof completions as code with the verified correctness guarantees of theorem provers, we can build systems that automate proof construction while maintaining rigorous correctness. Isabelle distinguishes itself from other provers through its use of higher-order logic as the underlying formalism, in contrast to tools such as Rocq and Lean, which are built on constructive type theory. Isabelle has been developed jointly by researchers at the University of Cambridge, UK, and the Technical University of Munich, Germany.

2. The Goal of the Project

The goal of this project is to fine-tune a large language model to successfully complete proofs in Isabelle. The input data will consist of a curated and verified repository of Isabelle proofs, augmented with structured output from Isabelle via an existing proof-of-concept client layer. Training will be carried out using Slurm on the University of Kent's cluster of NVIDIA A100 GPUs. The resulting model will be evaluated against published conference artifacts, and the evaluation will form the basis of a submission to a workshop in automated theorem proving and/or applied machine learning.

3. Technical Approach

The project proceeds in three broad phases: data preparation, model fine-tuning, and evaluation.

In the data preparation phase, an existing corpus of Isabelle proofs will be processed through a client layer that interfaces with the Isabelle proof assistant. This layer extracts structured proof states — including goals, hypotheses, and tactic applications — at each intermediate step, producing a dataset of (proof state, next tactic) pairs suitable for supervised fine-tuning.

In the fine-tuning phase, a pre-trained code-focused LLM will be adapted to the Isabelle proof domain using this dataset. Training will be conducted on the University of Kent's HPC cluster via Slurm, leveraging NVIDIA A100 GPUs. Standard parameter-efficient fine-tuning

techniques (e.g. LoRA) will be explored to reduce computational cost while preserving model quality.

In the evaluation phase, the fine-tuned model will be integrated into an agentic workflow proposing and verifying proof steps in real time by the Isabelle kernel. Performance will be assessed on a benchmark drawn from published conference artifacts, measuring proof completion rate and a qualitative measure of the proofs created.

4. Expected Outcomes

The primary deliverable is a fine-tuned LLM capable of generating valid Isabelle proof steps, integrated with the Isabelle proof assistant via the existing client layer. Secondary outcomes include a curated and reproducible training dataset of structured Isabelle proof traces, a documented evaluation benchmark drawn from conference artifacts, and a written evaluation of model performance. Collectively, these outputs are expected to support a co-authored submission to a workshop on automated theorem proving and/or applied machine learning.

5. Project Timeline

The project spans approximately three months, structured as follows:

Period	Activity
Month 1	Data pipeline development; corpus extraction and preprocessing via the Isabelle client layer
Month 2	Model fine-tuning and iterative experimentation on the HPC cluster
Month 3	Evaluation on conference benchmarks; write-up and preparation of workshop submission

6. Prerequisites and Required Skills

Applicants should have a solid foundation in machine learning and be comfortable working with LLMs, including fine-tuning workflows and associated tooling (e.g. Hugging Face Transformers). Proficiency in Python is essential. Experience with high-performance computing environments (e.g. Slurm) and Linux is desirable. Some familiarity with formal methods or functional programming is beneficial but not required — a willingness to engage with the underlying theory is more important than prior exposure.

7. Supervision and Publication

The student will be co-supervised throughout the project with regular weekly meetings by research and faculty staff. The project is explicitly research-oriented: a successful and timely implementation will form the basis of a co-authored submission to a systems and code

verification workshop, offering the student a concrete path to a peer-reviewed publication arising directly from their master's project work.

Any questions should be directed to the supervisors as early as possible.